ASP.NET

Unit-II: VB.NET Fundamentals

Comprehensive Study Notes

1. Introduction to VB.NET

1.1 What is VB.NET?

Visual Basic .NET (VB.NET) is an object-oriented programming language developed by Microsoft as part of the .NET Framework. It is a modern, type-safe, and fully object-oriented evolution of classic Visual Basic.

Key Characteristics of VB.NET

Feature	Description
Object-Oriented	Supports classes, inheritance, polymorphism, encapsulation
Event-Driven	Responds to user actions and system events
Type-Safe	Prevents type mismatches at compile time
Garbage Collected	Automatic memory management
Language Interoperable	Can use components written in other .NET languages

2. Basic Language Elements

2.1 Statements

A **statement** is a complete instruction in VB.NET that performs an action.

Types of Statements

```
' Declaration statement
Dim age As Integer = 25

' Assignment statement
name = "John"

' Method call statement
Console.WriteLine("Hello World")

' Multiple statements on same line (using colon)
Dim x As Integer = 10 : Dim y As Integer = 20 : Dim sum As Integer = x + y
```

Lines and Line Continuation

```
' Single line statement
Dim result As Integer = 10 + 20

' Line continuation using underscore (_)
Dim complexCalculation As Integer = _
    10 + 20 + 30 + 40

' Multiple statements on one line using colon
Dim a As Integer = 5 : Dim b As Integer = 10 : Console.WriteLine(a + b)
```

Comments

```
' This is a single-line comment
' Comments start with an apostrophe
Dim x As Integer = 10 ' Inline comment
```

```
''' <summary>
''' This is an XML documentation comment
''' </summary>
''' <param name="value">Input value</param>
''' <returns>Processed result</returns>
Function ProcessData(ByVal value As Integer) As Integer
End Function

' REM is an alternative way to write comments (obsolete but still works)
REM This is a comment using REM
```

3. Data Types

3.1 Value Types

Type	Size	Range / Description
Boolean	2 bytes	True or False
Byte	1 byte	0 to 255
Integer	4 bytes	-2,147,483,648 to 2,147,483,647
Long	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
Single	4 bytes	-3.402823E+38 to 3.402823E+38
Double	8 bytes	-1.79769313486232E+308 to 1.79769313486232E+308
Decimal	16 bytes	+/-79,228,162,514,264,337,593,543,950,335
Char	2 bytes	One Unicode character
Date	8 bytes	January 1, 0001 to December 31, 9999

3.2 Reference Types

Type	Description
String	Sequence of Unicode characters (immutable)
Array	Indexed collection of elements of the same type
Class	User-defined reference type (object-oriented)

Variable Declaration (Dim)

```
' Explicit declaration with data type
Dim name As String = "John Smith"
Dim age As Integer = 30
Dim salary As Decimal = 50000.50D
Dim isActive As Boolean = True
Dim hireDate As Date = #2024-01-01#

' Multiple declarations
Dim x As Integer, y As Integer, z As Integer

' Declaration without initialization
Dim temp As String
temp = "Value assigned later"

' Type inference (Option Infer On)
Dim inferredNum = 10    ' Type inferred as Integer
Dim inferredStr = "Hello" ' Type inferred as String
```

Constants

```
Const PI As Double = 3.14159
Const COMPANY_NAME As String = "Tech Solutions"
Const MAX_COUNT As Integer = 100
Const TAX_RATE As Decimal = 0.18D

' Using constants
Dim area As Double = PI * 5 ^ 2
Console.WriteLine($"Company: {COMPANY_NAME}")
```

4. Operators

4.1 Arithmetic Operators

Operator	Name	Example	Result
`+`	Addition	`10 + 5`	15
`-`	Subtraction	`10 - 5`	5
`*`	Multiplication	`10 * 5`	50
`/`	Division (Real)	`10 / 3`	3.333...
`\`	Integer Division	`10 \ 3`	3
`Mod`	Modulus	`10 Mod 3`	1
`^`	Exponentiation	`2 ^ 3`	8

```
' Arithmetic Operator Examples
Dim a As Integer = 10, b As Integer = 3
Console.WriteLine(a + b)  ' 13
Console.WriteLine(a - b)  ' 7
Console.WriteLine(a * b)  ' 30
Console.WriteLine(a / b)  ' 3.333...
Console.WriteLine(a \ b)  ' 3 (Integer division)
Console.WriteLine(a Mod b) ' 1 (Remainder)
Console.WriteLine(a ^ b)  ' 1000
```

4.2 Comparison Operators

Operator	Description	Example	Result
`=`	Equal to	`5 = 5`	True

`<>`	Not equal to	`5 <> 3`	True
`>`	Greater than	`5 > 3`	True
`<`	Less than	`5 < 3`	False
`>=`	Greater than or equal	`5 >= 5`	True
`<=`	Less than or equal	`5 <= 3`	False
`Is`	Object reference equality	`obj1 Is obj2`	Boolean
`IsNot`	Object reference inequality	`obj1 IsNot obj2`	Boolean

```
Dim age As Integer = 25
Dim isAdult As Boolean = (age >= 18) ' True
Dim isChild As Boolean = (age < 18)  ' False
```

4.3 Logical Operators

Operator	Description	Example	Result
`And`	Logical AND	`True And False`	False
`Or`	Logical OR	`True Or False`	True
`Not`	Logical NOT	`Not True`	False
`Xor`	Exclusive OR	`True Xor True`	False
`AndAlso`	Short-circuit AND	`c1 AndAlso c2`	Short-circuits
`OrElse`	Short-circuit OR	`c1 OrElse c2`	Short-circuits

```
Dim hasLicense As Boolean = True
Dim isAdult As Boolean = True
Dim canDrive As Boolean = isAdult And hasLicense ' True

' Short-circuit evaluation
```

```
Dim result As Boolean = (5 > 10) AndAlso (10 / 0 = 0) ' Doesn't evaluate
second part
```

4.4 Concatenation Operators

```
' & Operator - Preferred for strings
Dim fullName As String = "John" & " " & "Doe"

' + Operator - Can be ambiguous
Dim text As String = "Hello" + " World"

' Concatenation with numbers
Dim message As String = "Age: " & age.ToString()
```

4.5 Operator Precedence

```
' Order of evaluation (highest to lowest)
' 1. ^ (Exponentiation)
' 2. - (Negation)
' 3. *, /, \, Mod
' 4. +, -
' 5. & (Concatenation)
' 6. =, <>, >, <, >=, <=
' 7. Not
' 8. And, AndAlso
' 9. Or, OrElse, Xor

Dim result1 As Integer = (5 + 3) * 2 ' 16
Dim result2 As Integer = 5 + 3 * 2  ' 11 (multiplication first)
```

5. Control Structures

5.1 Branching (Decision Making)

If...ElseIf...Else

```
' Simple If
If age >= 18 Then
    Console.WriteLine("Adult")
End If

' If-Else
If age >= 18 Then
    Console.WriteLine("Adult")
Else
    Console.WriteLine("Minor")
End If

' If-ElseIf-Else
If age >= 60 Then
    Console.WriteLine("Senior")
ElseIf age >= 18 Then
    Console.WriteLine("Adult")
ElseIf age >= 13 Then
    Console.WriteLine("Teen")
Else
    Console.WriteLine("Child")
End If

' Multiple conditions
If age >= 18 And hasLicense Then
    Console.WriteLine("Can drive")
End If
```

Select Case

```
Select Case dayOfWeek
    Case 1
```

```
        Console.WriteLine("Monday")
    Case 2
        Console.WriteLine("Tuesday")
    Case 3 To 5
        Console.WriteLine("Midweek")
    Case 6, 7
        Console.WriteLine("Weekend")
    Case Else
        Console.WriteLine("Invalid")
End Select

' Select Case with comparison operators
Select Case score
    Case >= 90
        Console.WriteLine("A")
    Case >= 80
        Console.WriteLine("B")
    Case >= 70
        Console.WriteLine("C")
    Case Else
        Console.WriteLine("F")
End Select
```

5.2 Looping

For Loop

```
' Basic For loop
For i As Integer = 1 To 10 Step 1
    Console.WriteLine(i)
Next

' Step 2 (even numbers)
For i As Integer = 2 To 10 Step 2
    Console.WriteLine(i)
Next

' Reverse loop
For i As Integer = 10 To 1 Step -1
    Console.WriteLine(i)
Next
```

```
' Exit loop early
For i As Integer = 1 To 100
    If i = 50 Then Exit For
    Console.WriteLine(i)
Next
```

For Each Loop

```
' Array iteration
Dim fruits() As String = {"Apple", "Banana", "Orange"}
For Each fruit As String In fruits
    Console.WriteLine(fruit)
Next

' Dictionary iteration
Dim scores As New Dictionary(Of String, Integer)()
scores.Add("John", 85)
scores.Add("Jane", 92)
For Each kvp As KeyValuePair(Of String, Integer) In scores
    Console.WriteLine($"{kvp.Key}: {kvp.Value}")
Next
```

While Loop

```
' Basic While loop
Dim counter As Integer = 1
While counter <= 5
    Console.WriteLine(counter)
    counter += 1
End While

' Infinite loop with exit
Dim count As Integer = 0
While True
    count += 1
    If count >= 100 Then Exit While
End While
```

Do...Loop

```
' Do While (Check condition at start)
Dim i As Integer = 1
Do While i <= 5
    Console.WriteLine(i)
    i += 1
Loop

' Do Loop While (Check condition at end - executes at least once)
Dim j As Integer = 1
Do
    Console.WriteLine(j)
    j += 1
Loop While j <= 5

' Do Until (Run until condition becomes true)
Dim k As Integer = 1
Do Until k > 5
    Console.WriteLine(k)
    k += 1
Loop
```

6. Arrays

6.1 Single-Dimensional Arrays

```
Dim numbers(4) As Integer       ' 5 elements (0-4)
Dim names() As String = {"John", "Jane", "Bob"}
Dim scores As Integer() = {85, 90, 78, 92}

Console.WriteLine(scores(0))   ' 85
scores(1) = 95                 ' Modify
```

6.2 Multi-Dimensional Arrays

```
Dim matrix(2, 3) As Integer    ' 3 rows, 4 columns
matrix(0, 0) = 1
matrix(1, 2) = 5

' Multi-dimensional array iteration
Dim table(2, 2) As Integer
For i As Integer = 0 To 2
    For j As Integer = 0 To 2
        table(i, j) = i * j
    Next
Next
```

6.3 Dynamic Arrays (ReDim)

```
Dim dynamicArray() As Integer
ReDim dynamicArray(4)         ' Size 5
ReDim Preserve dynamicArray(9) ' Preserve values, increase size
```

6.4 Array Methods

```
Dim numbers() As Integer = {5, 2, 8, 1, 9, 3}
```

```
Array.Sort(numbers)           ' Sorting
Array.Reverse(numbers)        ' Reversing
Dim index As Integer = Array.IndexOf(numbers, 5) ' Searching
Array.Copy(numbers, copy, numbers.Length)        ' Copying
Array.Clear(numbers, 0, numbers.Length)          ' Clear
```

7. Procedures

7.1 Sub Procedures (No Return Value)

```
' Sub Procedure - Performs action, doesn't return value
Sub DisplayWelcome()
    Console.WriteLine("Welcome to VB.NET")
End Sub

' Sub with parameters
Sub ShowMessage(ByVal title As String, ByVal content As String)
    Console.WriteLine($"{title}: {content}")
End Sub

' Calling a Sub
DisplayWelcome()
ShowMessage("Error", "File not found!")
```

7.2 Function Procedures (Return Value)

```
' Function - Returns a value
Function CalculateArea(ByVal length As Double, ByVal width As Double) As
Double
    Return length * width
End Function

Function IsEven(ByVal number As Integer) As Boolean
    Return number Mod 2 = 0
End Function

' Calling a Function
Dim area As Double = CalculateArea(10, 5)
Dim isEven As Boolean = IsEven(10)
```

7.3 ByVal vs ByRef

```
' ByVal (copy) - Changes not reflected outside
' ByRef (reference) - Changes reflected outside

Sub ChangeValue(ByVal param1 As Integer, ByRef param2 As Integer)
    param1 = 100
    param2 = 200
End Sub

Dim a As Integer = 1, b As Integer = 2
ChangeValue(a, b)
' a remains 1, b becomes 200
```

7.4 Optional Parameters and ParamArray

```
' Optional parameters (must have default value)
Sub DisplayMessage(ByVal msg As String, Optional ByVal count As Integer = 1)

' Parameter arrays
Function SumAll(ByVal ParamArray numbers() As Integer) As Integer
    Dim total As Integer = 0
    For Each num In numbers
        total += num
    Next
    Return total
End Function

Dim result = SumAll(1, 2, 3, 4, 5) ' 15
```

7.5 Procedure Overloading

```
Function Add(ByVal a As Integer, ByVal b As Integer) As Integer
    Return a + b
End Function

Function Add(ByVal a As Double, ByVal b As Double) As Double
    Return a + b
End Function

Function Add(ByVal a As String, ByVal b As String) As String
```

```
    Return a & b
End Function
```

8. Object-Oriented Programming

8.1 Classes and Objects

```
' Class definition
Public Class Person
    ' Fields
    Private _name As String
    Private _age As Integer

    ' Constructor
    Public Sub New(ByVal name As String, ByVal age As Integer)
        _name = name
        _age = age
    End Sub

    ' Properties
    Public Property Name As String
        Get
            Return _name
        End Get
        Set(ByVal value As String)
            _name = value
        End Set
    End Property

    Public Property Age As Integer
        Get
            Return _age
        End Get
        Set(ByVal value As Integer)
            If value >= 0 Then _age = value
        End Set
    End Property

    ' Method
    Public Sub DisplayInfo()
        Console.WriteLine($"Name: {_name}, Age: {_age}")
    End Sub
```

```
End Class

' Using the class
Module Module1
    Sub Main()
        Dim p As New Person("John", 25)
        Console.WriteLine(p.Name)
        p.DisplayInfo()
    End Sub
End Module
```

8.2 Inheritance

```
' Base class
Public Class Animal
    Public Property Name As String

    Public Sub New(ByVal name As String)
        Me.Name = name
    End Sub

    Public Overridable Sub Speak()
        Console.WriteLine("Animal speaks")
    End Sub
End Class

' Derived class
Public Class Dog
    Inherits Animal

    Public Sub New(ByVal name As String)
        MyBase.New(name)
    End Sub

    Public Overrides Sub Speak()
        Console.WriteLine($"{Name} says Woof!")
    End Sub
End Class

' Using inheritance
Module Module1
    Sub Main()
```

```
        Dim d As New Dog("Buddy")
        d.Speak()
    End Sub
End Module
```

8.3 Polymorphism

```
Public Class Shape
    Public Overridable Function Area() As Double
        Return 0
    End Function
End Class

Public Class Circle
    Inherits Shape
    Private _radius As Double

    Public Sub New(ByVal r As Double)
        _radius = r
    End Sub

    Public Overrides Function Area() As Double
        Return Math.PI * _radius * _radius
    End Function
End Class

Public Class Rectangle
    Inherits Shape
    Private _width, _height As Double

    Public Sub New(ByVal w As Double, ByVal h As Double)
        _width = w
        _height = h
    End Sub

    Public Overrides Function Area() As Double
        Return _width * _height
    End Function
End Class
```

8.4 Accessibility Modifiers

Modifier	Description
`Public`	Accessible from anywhere
`Private`	Accessible only within the same class
`Protected`	Accessible within the class and derived classes
`Friend`	Accessible within the same assembly
`Protected Friend`	Protected + Friend combined

9. Key Differences to Remember

Concept	A	B
Passing	ByVal (copy)	ByRef (reference)
Procedure	Sub (no return)	Function (returns value)
Division	/ (real)	\ (integer)
Access	Public (all)	Private (same class)
Inheritance	Overridable	Overrides
Loop	For (fixed count)	While (condition)

Code Templates for Exams

Basic Class Template

```
Public Class ClassName
    ' Fields
    Private _field As DataType

    ' Constructor
    Public Sub New()
    End Sub

    ' Properties
    Public Property PropertyName As DataType
        Get
            Return _field
        End Get
        Set(ByVal value As DataType)
            _field = value
        End Set
    End Property
```

```
    ' Methods
    Public Sub MethodName()
    End Sub
End Class
```

Inheritance Template

```
Public Class BaseClass
    Public Overridable Sub MethodName()
    End Sub
End Class

Public Class DerivedClass
    Inherits BaseClass

    Public Overrides Sub MethodName()
        MyBase.MethodName()
    End Sub
End Class
```

Important Questions for Unit II

Short Answer Questions (2 marks)

1. What is the difference between `ByVal` and `ByRef`?
2. Define implicit and explicit variable declaration.
3. What are the different loop structures in VB.NET?
4. What is the difference between `Mod` and `\` operators?
5. What is an array? How to declare a multi-dimensional array?
6. What is the difference between `Sub` and `Function`?
7. What is inheritance in VB.NET?
8. What is the purpose of `Imports` statement?

Medium Answer Questions (5 marks)

1. Explain different data types in VB.NET with examples.
2. Explain the different types of arrays with examples.
3. What are the different accessibility modifiers? Explain with examples.
4. Explain the concepts of classes and objects with VB.NET code.
5. What is method overriding? How is it different from method overloading?

Long Answer Questions (12 marks)

1. Explain Object-Oriented Programming concepts (Abstraction, Encapsulation, Inheritance, Polymorphism) with VB.NET examples.
2. Write a complete VB.NET program demonstrating class inheritance with base and derived classes.
3. Explain string manipulation methods in VB.NET with examples.
4. What are namespaces? How do they relate to assemblies? Explain with examples.

Programming Questions (Frequently Asked)

1. Create a class `Student` with fields, properties, and methods.
2. Write a program to demonstrate polymorphism using method overriding.
3. Create a class library and use it in another application.

4. Write a program to demonstrate the use of `Select Case` with ranges.